

#5 SQL & Database Fundamentals

SQL and Database Fundamentals	RDBMS concepts, database design basics, primary and foreign keys, constraints, normalization basics, DDL, DML, DQL,
SQL and Database Fundamentals	filtering, sorting, aggregate functions, grouping, joins, subqueries, views, practical query-writing exercises

Database

A **database** is an organized collection of data that can be easily stored, managed, and retrieved.

Example

Instead of storing student records in Excel files:

Roll No	Name	Course
101	Ummed	Java
102	Bharat	Python

We store them in a database.

RDBMS

RDBMS (Relational Database Management System) stores data in the form of **tables** (relations).

Examples:

- MySQL
- PostgreSQL
- Oracle Database
- Microsoft SQL Server

Characteristics

- Data stored in tables
- Relationships between tables

- Uses SQL
- Supports constraints
- Reduces data redundancy

Relational vs non-relational databases

The main difference between **relational databases (SQL databases)** and **non-relational databases (NoSQL databases)** is how they store and organize data.

Feature	Relational Database (SQL)	Non-Relational Database (NoSQL)
Data Structure	Tables with rows and columns	Documents, key-value pairs, graphs, or wide-column stores
Schema	Fixed schema (predefined structure)	Flexible or dynamic schema
Relationships	Uses foreign keys and joins	Typically avoids joins; relationships handled differently
Scalability	Usually scales vertically (more powerful server)	Often scales horizontally (more servers)
Consistency	Strong consistency (ACID properties)	Often prioritizes availability and scalability (BASE model, depending on database)
Query Language	SQL	Database-specific APIs or query languages
Best For	Structured data and complex transactions	Large-scale, rapidly changing, or unstructured data

Non-Relational Database

A customer document might store everything together:

```
{  
  "customerId": 1,  
  "name": "Ummed",  
  "city": "Hyderabad",  
  "orders": [  

```

```
{  
  "orderId": 101,  
  "amount": 500  
}  
]  
}
```

Common non-relational databases:

- MongoDB
- Apache Cassandra
- Redis
- Neo4j

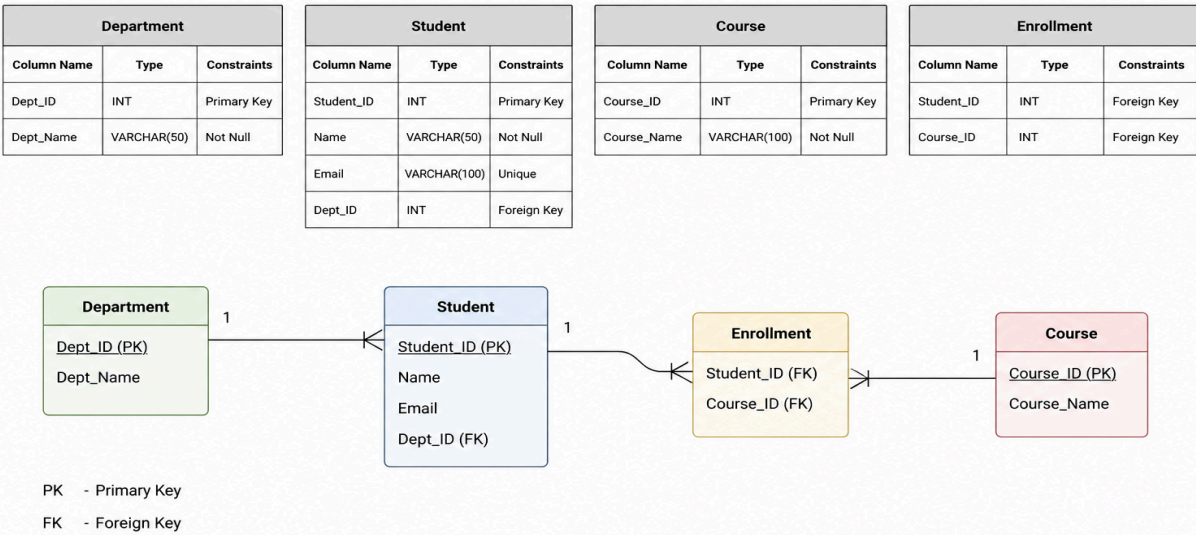
When to Use Which?

Choose a relational database if:

- Data has clear relationships.
- You need transactions (banking, e-commerce payments).
- Data integrity is critical.
- The structure doesn't change often.

Choose a non-relational database if:

- Data structure changes frequently.
 - You need to handle very large volumes of data.
 - High scalability and performance are priorities.
 - Data is semi-structured or unstructured (logs, social media, IoT).
-



Create Database

```
CREATE DATABASE CollegeDB;
```

Explanation

CREATE → Create something
DATABASE → Database object
CollegeDB → Database Name

Database Created Successfully.

Select Database

```
USE CollegeDB;
```

Why?

A MySQL server can contain many databases.

The USE command tells MySQL which database we want to work with.

Create Department Table

```
CREATE TABLE Department(  
    Dept_ID INT,
```

```
Dept_Name VARCHAR(50)  
);
```

Explanation

Column	Meaning
Dept_ID	Department ID
Dept_Name	Department Name

Data Types

INT: Stores numbers

- 1
- 2
- 3
- 100

VARCHAR(50): Stores text up to 50 characters

- Computer Science
- Mechanical
- Civil

Department Table Structure

Dept_ID	Dept_Name

Currently table is empty.

Insert Department Records

```
INSERT INTO Department  
VALUES(1, 'Computer Science');  
  
INSERT INTO Department  
VALUES(2, 'Mechanical');  
  
INSERT INTO Department  
VALUES(3, 'Civil');
```

View Data

```
SELECT * FROM Department;
```

Output:

Dept_ID	Dept_Name
1	Computer Science
2	Mechanical
3	Civil

Create Student Table

```
CREATE TABLE Student(  
    Student_ID INT,  
    Name VARCHAR(50),  
    Email VARCHAR(100),  
    Dept_ID INT  
);
```

Student Table Structure

Student_ID	Name	Email	Dept_ID
------------	------	-------	---------

Insert Student Records

```
INSERT INTO Student  
VALUES(101,'Rahul','rahul@gmail.com',1);  
  
INSERT INTO Student  
VALUES(102,'Priya','priya@gmail.com',2);  
  
INSERT INTO Student  
VALUES(103,'Amit','amit@gmail.com',1);
```

View Student Data

```
SELECT * FROM Student;
```

Output:

Student_ID	Name	Email	Dept_ID
101	Rahul	rahul@gmail.com	1
102	Priya	priya@gmail.com	2
103	Amit	amit@gmail.com	1

Note:

Question:

How do we know that Dept_ID = 1 means Computer Science?

Answer:

Because Dept_ID = 1 exists in the Department table.

This means both tables are connected.

SQL Data Types

In SQL, each column must be assigned a data type that defines the kind of data it can store, such as integers, dates, text or binary values. Choosing the correct data type is important for data integrity, query performance and efficient indexing.

1. Numeric Data Types

Numeric data types are used to store integer and decimal values. They support mathematical operations and are commonly used for financial, scientific and analytical data.

Exact Numeric Datatype

Exact numeric types are used when precise numeric values are needed, such as for financial data, quantities and counts. Some common exact numeric types include:

Data Type	Description	Range
BIGINT	Large integer numbers	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
INT	Standard integer values	-2,147,483,648 to 2,147,483,647
SMALLINT	Small integers	-32,768 to 32,767
TINYINT	Very small integers	0 to 255
DECIMAL	Exact fixed-point numbers (e.g., for financial values)	$-10^{38} + 1$ to $10^{38} - 1$
NUMERIC	Similar to DECIMAL, used for precision data	$-10^{38} + 1$ to $10^{38} - 1$

Example:

```
CREATE TABLE Product_Sales (  
    ProductID INT PRIMARY KEY,  
    Quantity SMALLINT,  
    UnitPrice DECIMAL(10,2),  
    TotalAmount DECIMAL(10,2)  
);
```

Approximate Numeric Datatype

These types are used to store approximate values, such as scientific measurements or large ranges of data that don't need exact precision.

Data Type	Description	Range
FLOAT	Approximate numeric values	-1.79E+308 to 1.79E+308
REAL	Similar to FLOAT, but with less precision	-3.40E+38 to 3.40E+38

Example:

```
CREATE TABLE Measurements (  
    SensorID INT,  
    Temperature FLOAT,  
    Humidity REAL  
);
```

Data Type	Description
CHAR	Stores fixed-length non-Unicode characters with a maximum length of 8000 characters

VARCHAR	Stores variable-length non-Unicode characters with a maximum length of 8000 characters
VARCHAR(MAX)	Stores variable-length non-Unicode data with a maximum size of $2^{31} - 1$ characters (introduced in SQL Server 2005)
TEXT	Stores variable-length non-Unicode data with a maximum size of 2,147,483,647 characters

2. Character and String Data Types

Character data types are used to store text or character-based data. The choice between fixed-length and variable-length data types depends on the nature of your data.

Example:

```
CREATE TABLE Employee_Info (  
    EmpID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName CHAR(30),  
    Bio Nvarchar(max)  
);
```

Unicode Character String Data Types

Unicode data types are used to store characters from any language, supporting a wider variety of characters. These are given in below table.

Data Type	Description
Nchar	The maximum length of 4000 characters(Fixed-Length Unicode Characters)
Nvarchar	The maximum length of 4000 characters.(Variable-Length Unicode Characters)
Nvarchar(max)	The maximum length of $2^{31} - 1$ characters(SQL Server 2005 only). (Variable Length Unicode data)

Example:

```
CREATE TABLE International_Users (  
  
    UserID INT PRIMARY KEY,  
  
    FullName NVARCHAR(100),  
  
    Country NCHAR(50)  
  
);
```

3. Date and Time Data Type

SQL provides several data types for storing date and time information. They are essential for managing timestamps, events and time-based queries. These are given in the below table.

Data Type	Description	Storage Size
DATE	Stores the data of date (year, month, day)	3 Bytes

TIME	Stores the data of time (hour, minute,second)	3 Bytes
DATETIME	Stores both the data and time (year, month, day, hour, minute, second)	8 Bytes

Example:

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    OrderDate DATE,  
    OrderTime TIME,  
    ShippedAt DATETIME  
);
```

4. Binary Data Types in SQL

Binary data types are used to store binary data such as images, videos or other file types. These include

Data Type	Description	Max Length
Binary	Fixed-length binary data.	8000 bytes
VarBinary	Variable-length binary data.	8000 bytes
Image	Stores binary data as images.	2,147,483,647 bytes

Example:

```
CREATE TABLE Product_Images (  
    ImageID INT PRIMARY KEY,  
    ImageName VARCHAR(100),
```

```
ImageData VARBINARY (MAX)  
);
```

5. Boolean Data Type in SQL

Boolean data types store logical values such as TRUE and FALSE. In SQLite, these values are typically represented using integers. where:

- 1 represents TRUE
- 0 represents FALSE

Example:

```
CREATE TABLE User_Status (  
    UserID INT PRIMARY KEY,  
    IsActive INTEGER,  
    IsVerified INTEGER  
);
```

6. Special Data Types

SQL also supports some specialized data types for advanced use cases:

- **XML Data Type:** Used to store XML data and manipulate XML structures in the database

Example:

```
CREATE TABLE XML_Records (  
    RecordID INT PRIMARY KEY,  
    ConfigData XML  
);
```

- **Spatial Data Type (Geometry):** stores planar spatial data, such as points, lines and polygons, in a database table.

Example:

```
CREATE TABLE Locations (  
  
    LocationID INT PRIMARY KEY,  
  
    Area GEOMETRY  
  
);
```

Database Design

Entity?

An Entity is a real-world object about which data is stored.

Examples

- Student
- Department
- Course
- Faculty
- Employee

Attribute

An Attribute is a property of an entity.

Student Entity

- Student_ID
- Name
- Email

Department Entity

- Dept_ID
 - Dept_Name
-

Relationship Between Entities

Entities can be related to each other.

One-to-One (1:1)

Person
|
Passport

One person has one passport.

One-to-Many (1:M)

Department
|
Student

One department can have many students.

Example:

Computer Science
|
|— Rahul
|— Amit
|— Neha

Many-to-Many (M:M)

Student
↔
Course

One student can enroll in many courses.

One course can have many students.

Example:

Rahul → Java

Rahul → Python

Java → Rahul

Java → Priya

Primary Key

A Primary Key uniquely identifies each row.

Properties

- Unique
- Cannot be NULL
- One per table

Example

Student Table

Student_ID	Name
101	Ummed
102	Bharat

```
Student_ID = Primary Key
```

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50)  
);
```

Types of PRIMARY KEYS:

There are two types of primary keys:

1. **Simple Primary Key:** A primary key that consists of a single column.
2. **Composite Primary Key:** A primary key that consists of multiple columns.

2. Create a Composite Primary Key (Multiple Columns)

If you need multiple columns to combine to form a unique identifier for a row, you can create a single primary key made up of multiple fields. This is called a composite primary key.

For example, in an Order Details table, an individual `orderId` can repeat, and an individual `ProductID` can repeat, but the combination of both must be unique.

```
CREATE TABLE OrderDetails (  
    OrderID INT,  
    ProductID INT,  
    Quantity INT,  
    -- This defines ONE primary key that spans TWO columns  
    PRIMARY KEY (OrderID, ProductID)  
);
```

2. Use a Unique Constraint for Alternate Keys

If you want to ensure that completely separate columns are unique (for example, you want `EmployeeID` as the primary key, but you also want to ensure that `Email` and `SocialSecurityNumber` cannot be duplicated), you should use a `UNIQUE` constraint.

A table can have only one primary key, but it can have multiple unique constraints.

sql

```
CREATE TABLE Employees (  
    EmployeeID INT NOT NULL,  
    Email VARCHAR(100) NOT NULL,  
    PhoneNumber VARCHAR(15),
```

```
-- Primary Key Constraint

        CONSTRAINT PK_Employee PRIMARY KEY (EmployeeID),

-- Additional unique constraints acting like secondary keys

        CONSTRAINT UQ_Employee_Email UNIQUE (Email),

        CONSTRAINT UQ_Employee_Phone UNIQUE (PhoneNumber)

);
```

Foreign Key

A Foreign Key creates a relationship between tables.

Example

Department Table

Dept_ID	Dept_Name
1	CSE
2	IT

Student Table

Student_ID	Name	Dept_ID
101	Ummed	1

Dept_ID in Student table refers to Department table.

Syntax

```
FOREIGN KEY (child_column)
```

```
REFERENCES parent_table(parent_column)
```

Example

```
CREATE TABLE Department(  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50)  
);
```

```
CREATE TABLE Student(  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Dept_ID INT,  
    FOREIGN KEY (Dept_ID)  
    REFERENCES Department(Dept_ID)  
);
```

OR

```
CREATE TABLE Student(  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Dept_ID INT REFERENCES Department(Dept_ID)  
);
```

SQL Commands Categories

DDL (Data Definition Language)

Used to define database structure.

Commands:

- CREATE
- ALTER
- DROP
- TRUNCATE

CREATE

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50)  
);
```

ALTER

Add column

```
ALTER TABLE Student  
ADD Email VARCHAR(100);
```

DROP

Delete table permanently.

```
DROP TABLE Student;
```

TRUNCATE

Delete all records.

```
TRUNCATE TABLE Student;
```

DML (Data Manipulation Language)

Used to manipulate data.

Commands:

- INSERT
- UPDATE
- DELETE

INSERT

```
INSERT INTO Student  
VALUES (101, 'Ummed');
```

UPDATE

```
UPDATE Student  
SET Name='Ummed'  
WHERE Student_ID=101;
```

DELETE

```
DELETE FROM Student  
WHERE Student_ID=101;
```

DQL (Data Query Language)

Used to retrieve data.

Command:

SELECT

Fetch All Records

```
SELECT * FROM Student;
```

Specific Columns

```
SELECT Name, Email  
FROM Student;
```

Constraints

- Constraints enforce rules on data.

PRIMARY KEY

```
Student_ID INT PRIMARY KEY
```

-
- Must be unique.

NOT NULL

```
Name VARCHAR(50) NOT NULL
```

-
- Cannot store NULL values.

UNIQUE

```
Email VARCHAR(100) UNIQUE
```

-
- No duplicate values allowed.

DEFAULT

```
Status VARCHAR(20) DEFAULT 'Active'
```

CHECK

```
Age INT CHECK (Age >= 18)
```

FOREIGN KEY

```
FOREIGN KEY (Dept_ID)  
REFERENCES Department (Dept_ID)
```

Filtering Data (WHERE Clause)

Suppose:

ID	Name	Age
1	Ummed	25
2	Yashpal	26
3	Nikhil	25

Equal To

```
SELECT *  
FROM Student  
WHERE Age = 25;
```

Greater Than

```
SELECT *  
FROM Student  
WHERE Age > 18;
```

BETWEEN

```
SELECT *  
FROM Student  
WHERE Age BETWEEN 18 AND 22;
```

IN

```
SELECT *  
FROM Student  
WHERE Age IN(18,20);
```

LIKE

```
SELECT *  
FROM Student  
WHERE Name LIKE 'R%';
```

Starts with R

Sorting Data

Ascending

```
SELECT *  
FROM Student  
ORDER BY Name ASC;
```

Descending

```
SELECT *  
FROM Student  
ORDER BY Name DESC;
```

Aggregate Functions

Used to perform calculations.

Assume Employee table.

Salary
20000
30000
40000

COUNT

```
SELECT COUNT(*)  
FROM Employee;
```

SUM

```
SELECT SUM(Salary)  
FROM Employee;
```

AVG

```
SELECT AVG(Salary)
FROM Employee;
```

MAX

```
SELECT MAX(Salary)
FROM Employee;
```

MIN

```
SELECT MIN(Salary)
FROM Employee;
```

GROUP BY

Used to group records.

Employee Table

Department	Salary
IT	30000
IT	40000
HR	25000

Average Salary by Department

```
SELECT Department,
       AVG(Salary)
FROM Employee
GROUP BY Department;
```

Output

Department	AVG
------------	-----

IT	35000
HR	25000

HAVING Clause

Filters grouped data.

```
SELECT Department,  
       AVG(Salary)  
FROM Employee  
GROUP BY Department  
HAVING AVG(Salary) > 30000;
```

SQL Joins

Most important interview topic.

Sample Tables

Student

Student_ID	Name	Dept_ID
1	Ummed	10
2	Priya	20

Department

Dept_ID	Dept_Name
10	CSE
20	IT

INNER JOIN

Returns matching rows.

```
SELECT s.Name,  
       d.Dept_Name  
FROM Student s  
INNER JOIN Department d  
ON s.Dept_ID = d.Dept_ID;
```

Output

Name	Dept_Name
Ummed	CSE
Priya	IT

LEFT JOIN

All rows from left table.

```
SELECT *  
FROM Student s  
LEFT JOIN Department d  
ON s.Dept_ID=d.Dept_ID;
```

RIGHT JOIN

All rows from right table.

```
SELECT *  
FROM Student s  
RIGHT JOIN Department d  
ON s.Dept_ID=d.Dept_ID;
```

FULL JOIN

All rows from both tables.

```
SELECT *  
FROM Student s
```

```
FULL JOIN Department d  
ON s.Dept_ID=d.Dept_ID;
```

Subqueries

Query inside another query.

Example

Find employee with highest salary.

```
SELECT *  
FROM Employee  
WHERE Salary =  
(  
    SELECT MAX(Salary)  
    FROM Employee  
);
```

Employees Above Average Salary

```
SELECT *  
FROM Employee  
WHERE Salary >  
(  
    SELECT AVG(Salary)  
    FROM Employee  
);
```

Views

Virtual table based on query.

Create View

```
CREATE VIEW Student_View AS
```

```
SELECT Student_ID,  
       Name  
FROM Student;
```

Use View

```
SELECT *  
FROM Student_View;
```

Advantages

- Security
 - Simplifies complex queries
 - Reusability
-

Normalization

Normalization removes redundancy and improves consistency.

Unnormalized Table

Student_ID	Student_Name	Courses
101	Ummed	Java,Python

Problem:

Multiple values in one column.

First Normal Form (1NF)

- Each column should contain a single value.

Student_ID	Student_Name	Course
101	Ummed	Java
101	Ummed	Python

Second Normal Form (2NF)

- Remove partial dependency.
- Separate student information and course information.

Student

Student_ID	Student_Name
------------	--------------

Course

Course_ID	Course_Name
-----------	-------------

Enrollment

Student_ID	Course_ID
------------	-----------

Third Normal Form (3NF)

Remove transitive dependency.

Bad Design

Emp_ID	Dept_ID	Dept_Name
--------	---------	-----------

Good Design

Employee Table

Emp_ID	Dept_ID
--------	---------

Department Table

Dept_ID	Dept_Name
---------	-----------

Practical Query Writing Exercises

1. Create Student Table

```
CREATE TABLE Student(  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

2. Insert 5 Records

```
INSERT INTO Student  
VALUES  
    (1, 'Ummed', 25),  
    (2, 'Priya', 24),  
    (3, 'Aman', 26),  
    (4, 'Neha', 25),  
    (5, 'Karan', 24);
```

3. Students Older Than 20

```
SELECT *  
FROM Student  
WHERE Age > 24;
```

4. Sort by Name

```
SELECT *  
FROM Student  
ORDER BY Name;
```

5. Count Students

```
SELECT COUNT(*)  
FROM Student;
```

6. Average Age

```
SELECT AVG(Age)  
FROM Student;
```

7. Department-wise Student Count

```
SELECT Dept_ID,  
    COUNT(*)  
FROM Student  
GROUP BY Dept_ID;
```

8. Join Student and Department

```
SELECT s.Name,  
       d.Dept_Name  
FROM Student s  
INNER JOIN Department d  
ON s.Dept_ID=d.Dept_ID;
```

9. Student in CSE Department

```
SELECT s.Name  
FROM Student s  
JOIN Department d  
ON s.Dept_ID=d.Dept_ID  
WHERE d.Dept_Name='CSE';
```

10. Students Above Average Age

```
SELECT *  
FROM Student  
WHERE Age >  
(  
    SELECT AVG(Age)  
    FROM Student  
);
```

Interview Questions

1. Difference between DBMS and RDBMS?
2. What is a Primary Key?
3. What is a Foreign Key?
4. Difference between DELETE, DROP, and TRUNCATE?
5. What is Normalization?
6. Difference between WHERE and HAVING?
7. Difference between INNER JOIN and LEFT JOIN?
8. What is a View?
9. What is a Subquery?
10. What are Aggregate Functions?